

Geometry Optimisation Module

Mathematical Formulation and Script Implementation

Based on `optimizationModule.py` by Tiago Cardoso (FEUP, 2025)

1 Overview

The geometry optimisation module is a standalone Python tool that iteratively modifies the cross-sectional geometry of a composite crush structure to meet specific performance objectives. It operates on cross-section data generated by a MATLAB-based analytical crush prediction tool, receiving the nodal coordinates and connectivity of a single-layer cross-section.

The module supports two optimisation modes:

- (i) **Maximise crush force** — adjust nodal positions to increase the total crush force developed by the cross-section.
- (ii) **Minimise perimeter** — reduce the cross-section perimeter (and hence structural mass) while maintaining the crush force above its initial value.

The optimisation is implemented using the PyTorch deep learning framework, which provides automatic differentiation of the objective function with respect to the optimisation variables, and the Adam optimiser for iterative parameter updates.

2 Geometric Representation

2.1 Coordinate System

The cross-section is defined by a set of N ordered nodes $\{\mathbf{p}_i\}_{i=1}^N$ connected by edges to form a closed contour. These nodes are initially specified in Cartesian coordinates (x_i, y_i, z_i) and are transformed into spherical coordinates (r_i, θ_i, ϕ_i) relative to the centroid $\bar{\mathbf{p}}$ of the cross-section:

$$\bar{\mathbf{p}} = \frac{1}{N} \sum_{i=1}^N \mathbf{p}_i \quad (1)$$

The transformation from Cartesian to spherical coordinates, applied to the centroid-relative positions $\mathbf{q}_i = \mathbf{p}_i - \bar{\mathbf{p}}$, is given by:

$$r_i = \sqrt{q_{x,i}^2 + q_{y,i}^2 + q_{z,i}^2} \quad (2)$$

$$\theta_i = \arccos\left(\frac{q_{z,i}}{r_i}\right) \quad (3)$$

$$\phi_i = \text{atan2}(q_{y,i}, q_{x,i}) \quad (4)$$

where r_i is the radial distance, θ_i is the polar angle, and ϕ_i is the azimuthal angle.

In the script, this is implemented in the `Cartesian2Spherical()` function and is called during model initialisation:

```
1 r = torch.sqrt(X**2 + Y**2 + Z**2)
2 theta = torch.acos(Z / r)
3 phi = torch.atan2(Y, X)
```

2.2 Optimisation Variable

The primary optimisation variable is the radial distance r_i of each node from the centroid. The angular coordinates θ_i and ϕ_i are held constant throughout the optimisation. This means the optimiser only modifies how far each node sits from the centre, changing the overall shape and local curvature while preserving the angular distribution:

$$\Theta = \{r_1, r_2, \dots, r_N\} \quad (\text{learnable parameters}) \quad (5)$$

In PyTorch, this is achieved by declaring r as an `nn.Parameter` (tracked by the autograd engine and updated by the optimiser), while θ and ϕ are registered as non-learnable buffers:

```
1 self.r = nn.Parameter(r)           # learnable
2 self.register_buffer('theta', theta) # fixed
3 self.register_buffer('phi', phi)    # fixed
```

2.3 Cartesian Reconstruction

At each optimisation step (forward pass), the current radial distances are converted back to Cartesian coordinates using the inverse transformation:

$$x_i = r_i \sin \theta_i \cos \phi_i \quad (6)$$

$$y_i = r_i \sin \theta_i \sin \phi_i \quad (7)$$

$$z_i = r_i \cos \theta_i \quad (8)$$

The centroid is then added back to recover the absolute node positions:

$$\mathbf{p}_i = (x_i, y_i, z_i) + \bar{\mathbf{p}} \quad (9)$$

3 Radius of Curvature Computation

The radius of curvature (RoC) at each node is the central geometric quantity driving the optimisation. Lower RoC leads to higher crush stress through the curvature factor equation (Section ??). The computation proceeds through several stages.

3.1 Overlap Extension for Closed Loops

Since the cross-section is a closed contour, accurate spline fitting and derivative computation at the boundary nodes (first and last in the sequence) requires special treatment. The overlap technique wraps n_{overlap} nodes from each end of the sequence to the opposite end, creating an extended node list:

$$\{\mathbf{p}_i\}_{\text{ext}} = \underbrace{\{\mathbf{p}_{N-n_{\text{ov}}+1}, \dots, \mathbf{p}_N\}}_{\text{tail overlap}}, \mathbf{p}_1, \dots, \mathbf{p}_N, \underbrace{\{\mathbf{p}_1, \dots, \mathbf{p}_{n_{\text{ov}}}\}}_{\text{head overlap}} \quad (10)$$

where n_{ov} denotes the overlap count (set to 7 in the script). This ensures that the spline “sees” the continuity across the closure when fitting and computing derivatives near the endpoints. In the script, this is handled by the `nodesCoords()` function:

```
1 X_ext = torch.cat([X[-overlapCount:], X, X[:overlapCount]], dim=0)
2 Y_ext = torch.cat([Y[-overlapCount:], Y, Y[:overlapCount]], dim=0)
3 Z_ext = torch.cat([Z[-overlapCount:], Z, Z[:overlapCount]], dim=0)
```

3.2 Chord-Length Parametrisation

The extended nodes are parametrised by cumulative chord length to provide a natural parameter for spline fitting. For M extended nodes, the chord-length parameter s is:

$$s_1 = 0, \quad s_j = \sum_{k=1}^{j-1} \|\mathbf{p}_{k+1}^{\text{ext}} - \mathbf{p}_k^{\text{ext}}\|, \quad j = 2, \dots, M \quad (11)$$

This is then normalised to the unit interval:

$$\hat{s}_j = \frac{s_j}{s_M}, \quad \hat{s} \in [0, 1] \quad (12)$$

The `chordLength()` function implements this:

```
1 diffs = nodesExt[1:] - nodesExt[:-1]
2 ds = torch.norm(diffs, dim=1)
3 s = torch.zeros(len(nodesExt))
4 s[1:] = torch.cumsum(ds, dim=0)
5 s = s / s[-1]
```

3.3 Natural Cubic Spline Fitting

A natural cubic spline $\mathbf{C}(\hat{s})$ is fitted through the extended node sequence, mapping the scalar parameter $\hat{s}$ to 3D coordinates (x, y, z) :

$$\mathbf{C}(\hat{s}) : [0, 1] \rightarrow \mathbb{R}^3 \quad (13)$$

The fitting uses the `torchcubicspline` library, which provides a PyTorch-compatible implementation that supports automatic differentiation through the spline coefficients. This is critical: because the spline fitting is part of the computational graph, PyTorch can propagate gradients from the loss function back through the RoC calculation to the learnable radial distances.

The first derivative $\mathbf{C}'(\hat{s})$ is obtained analytically from the spline coefficients. The second derivative $\mathbf{C}''(\hat{s})$ is approximated via central finite differences:

$$\mathbf{C}''(\hat{s}) \approx \frac{\mathbf{C}'(\hat{s} + \delta) - \mathbf{C}'(\hat{s} - \delta)}{2\delta}, \quad \delta = 10^{-5} \quad (14)$$

In the script:

```
1 coeffs = natural_cubic_spline_coeffs(s, nodesExt)
2 spline = NaturalCubicSpline(coeffs)
3 splineDer1 = spline.derivative(sQuery)
4 # Central finite difference for second derivative
5 d1_plus = spline.derivative(sQuery + delta)
6 d1_minus = spline.derivative(sQuery - delta)
7 splineDer2 = (d1_plus - d1_minus) / (2 * delta)
```

3.4 Curvature and Radius of Curvature

The curvature κ_i at each node is computed using the standard formula for the curvature of a space curve:

$$\kappa_i = \frac{\|\mathbf{C}'(\hat{s}_i) \times \mathbf{C}''(\hat{s}_i)\|}{\|\mathbf{C}'(\hat{s}_i)\|^3} \quad (15)$$

where $\times$ denotes the vector cross product. The radius of curvature is then:

$$R_i = \frac{1}{\kappa_i} \quad (16)$$

A lower bound is applied to prevent numerical instabilities from extremely high curvatures:

$$R_i \leftarrow \max(R_i, 5) \quad (17)$$

The implementation in `RoCCalc()`:

```
1 cross = torch.cross(splineDer1, splineDer2, dim=1)
2 num = torch.norm(cross, dim=1)
3 den = torch.norm(splineDer1, dim=1) ** 3
4 curvature = num / den
5 radius = 1.0 / curvature
```

4 Curvature–Crush Stress Relationship

The influence of local curvature on crush stress is captured by a curvature factor f_{RoC} , which multiplies the constant (flat-plate) crush stress σ_{flat} to give the local nodal crush stress:

$$\sigma_{\text{crush},i} = \sigma_{\text{flat}} \cdot f_{\text{RoC}}(R_i) \quad (18)$$

The curvature factor used in the script is a fitted power-law relationship derived from CFRP laminate data:

$$f_{\text{RoC}}(R) = 7.95 R^{-0.94} + 0.99 \quad (19)$$

This function has the physical property that $f_{\text{RoC}} \rightarrow \infty$ as $R \rightarrow 0$ (very tight curves produce very high crush stress) and $f_{\text{RoC}} \rightarrow 0.99$ as $R \rightarrow \infty$ (flat regions produce approximately the baseline crush stress). The lower clamp on R at 5 mm prevents the power law from producing unrealistically large stress factors.

The implementation:

```
1 def CurvatureEq(x):
2     return 7.95 * torch.pow(x, -0.94) + 0.99
```

5 Perimeter Computation

The perimeter P is the total length of the closed contour, computed as the sum of Euclidean distances between consecutive ordered nodes:

$$P = \sum_{i=1}^N \|\mathbf{p}_{(i \bmod N)+1} - \mathbf{p}_i\| \quad (20)$$

The use of modular indexing ensures that the edge from the last node back to the first is included. In the script, this is elegantly handled with `torch.roll`:

```
1 def perimeterCalc(orderedNodes):
2     nodes_shifted = torch.roll(orderedNodes, -1, dims=0)
3     diffs = nodes_shifted - orderedNodes
4     return torch.sum(torch.norm(diffs, dim=1))
```

6 Crush Force Computation

The total crush force of the layer is computed as:

$$F_{\text{crush}} = t \cdot P \cdot \bar{\sigma}_{\text{crush}} \quad (21)$$

where t is the shell thickness, P is the perimeter (Eq. ??), and $\bar{\sigma}_{\text{crush}}$ is the mean crush stress across all nodes:

$$\bar{\sigma}_{\text{crush}} = \frac{1}{N} \sum_{i=1}^N \sigma_{\text{crush},i} = \frac{\sigma_{\text{flat}}}{N} \sum_{i=1}^N f_{\text{RoC}}(R_i) \quad (22)$$

This simplified formulation assumes a uniform layup (constant thickness) across the entire cross-section and considers only the radius of curvature as the factor influencing crush stress. In the script:

```
1 crushStress = self.constantCrushStress * CurvatureEq(RoC)
2 perimeter = perimeterCalc(orderedNodes)
3 forceTotal = self.thickness * perimeter * crushStress.mean()
```

7 Objective Functions and Loss Formulation

Since PyTorch optimisers perform minimisation, the objective functions are formulated as loss functions $\mathcal{L}$ to be minimised, augmented with penalty terms to enforce constraints.

7.1 Case 1: Maximise Crush Force

To maximise F_{crush} , the loss is defined as its negation, combined with a smoothness penalty:

$$\mathcal{L}_{\text{force}} = -F_{\text{crush}} + \lambda_{\text{smooth}} \cdot \mathcal{P}_{\text{smooth}} \quad (23)$$

Minimising $\mathcal{L}_{\text{force}}$ is equivalent to maximising F_{crush} subject to the smoothness constraint.

7.2 Case 2: Minimise Perimeter

To reduce the structural mass via perimeter reduction while maintaining crush performance:

$$\mathcal{L}_{\text{perim}} = P + \lambda_{\text{smooth}} \cdot \mathcal{P}_{\text{smooth}} + \lambda_{\text{force}} \cdot \mathcal{P}_{\text{force}} \quad (24)$$

7.3 Case 3: Minimise Perimeter with Inward Constraint

When nodes must remain within the initial cross-section boundary (e.g., for confined-space applications):

$$\mathcal{L}_{\text{perim,inward}} = P + \lambda_{\text{smooth}} \cdot \mathcal{P}_{\text{smooth}} + \lambda_{\text{force}} \cdot \mathcal{P}_{\text{force}} + \lambda_{\text{radial}} \cdot \mathcal{P}_{\text{radial}} \quad (25)$$

The selection between these cases is controlled by Boolean flags in the script:

```
1 if minPerimeterFlag and not maxCrushForceFlag:
2     if inwardSecFlag:
3         loss = perimeter + lam_s * smooth + lam_f * crush + lam_r *
          radial
4     else:
5         loss = perimeter + lam_s * smooth + lam_f * crush
6 elif maxCrushForceFlag and not minPerimeterFlag:
7     loss = -force_total + lam_s * smooth
```

8 Penalty Functions

Three penalty functions enforce constraints on the optimisation. Each is multiplied by a penalty weight λ that controls its strength relative to the primary objective. In all cases tested, $\lambda = 10^{10}$.

8.1 Smoothness Penalty $\mathcal{P}_{\text{smooth}}$

This penalty ensures geometric smoothness by penalising large variations in the radial distance profile. It approximates the squared second derivative of r along the contour:

$$\mathcal{P}_{\text{smooth}} = \sum_{i=2}^{N-1} (\Delta^2 r_i)^2 = \sum_{i=2}^{N-1} (r_{i+1} - 2r_i + r_{i-1})^2 \quad (26)$$

where $\Delta^2 r_i = r_{i+1} - 2r_i + r_{i-1}$ is the second finite difference. Large values of $\Delta^2 r$ indicate a sharp change in radial distance, corresponding to a corrugated or jagged cross-section shape. The implementation:

```
1 def smoothnessTerm(r):
2     dr = r[1:] - r[:-1]
3     curvature = dr[1:] - dr[:-1] # second difference
4     smoothness_loss = torch.sum(curvature ** 2)
5     return smoothness_loss
```

8.2 Crush Force Constraint $\mathcal{P}_{\text{force}}$

Applied only during perimeter minimisation, this penalty activates when the crush force drops below the initial value F_{crush}^0 :

$$\mathcal{P}_{\text{force}} = \left[\text{ReLU}(F_{\text{crush}}^0 - F_{\text{crush}}) \right]^2 \quad (27)$$

where the Rectified Linear Unit is defined as:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \quad (28)$$

This means the penalty is zero when $F_{\text{crush}} \geq F_{\text{crush}}^0$ (constraint satisfied) and grows quadratically with the deficit when the force falls below the initial value. In the script:

```
1 crushForce_loss = torch.relu(forceTotal_initial - force_total)**2
```

8.3 Radial Distance Constraint $\mathcal{P}_{\text{radial}}$

Applied only when enforcing inward-only node displacement, this penalty activates when any node moves outward beyond its initial radial distance r_i^0 :

$$\mathcal{P}_{\text{radial}} = \sum_{i=1}^N \left[\text{ReLU}(r_i - r_i^0) \right]^2 \quad (29)$$

The penalty is zero for all nodes that have moved inward ($r_i \leq r_i^0$) and quadratic for nodes that have moved outward. In the script:

```
1 radialDist_loss = torch.sum(torch.relu(model.r - model.initial_r)**2)
```

9 Optimisation Algorithm

9.1 Adam Optimiser

The Adam (Adaptive Moment Estimation) algorithm is used to update the radial distances. Adam maintains per-parameter adaptive learning rates based on estimates of the first moment

(mean) m_t and second moment (uncentred variance) v_t of the gradients. The update rule at iteration t is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (30)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (31)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (32)$$

$$r_i^{(t+1)} = r_i^{(t)} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \quad (33)$$

where $g_t = \partial \mathcal{L} / \partial r_i$ is the gradient, η is the base learning rate (default: 0.1), $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

9.2 Computational Graph and Automatic Differentiation

A key feature of this implementation is that the entire pipeline — from radial distances through Cartesian reconstruction, spline fitting, RoC computation, crush stress evaluation, and loss calculation — is built as a single PyTorch computational graph. This enables automatic differentiation: calling `loss.backward()` traverses the graph in reverse, computing:

$$\frac{\partial \mathcal{L}}{\partial r_i} = \frac{\partial \mathcal{L}}{\partial F_{\text{crush}}} \cdot \frac{\partial F_{\text{crush}}}{\partial \bar{\sigma}_{\text{crush}}} \cdot \frac{\partial \bar{\sigma}_{\text{crush}}}{\partial R_i} \cdot \frac{\partial R_i}{\partial \mathbf{C}'} \cdot \frac{\partial \mathbf{C}'}{\partial \mathbf{p}_i} \cdot \frac{\partial \mathbf{p}_i}{\partial r_i} + \dots \quad (34)$$

for every node i simultaneously, via the chain rule. This eliminates the need for manual differentiation of the complex objective function.

9.3 Optimisation Loop

The complete optimisation procedure is summarised in Algorithm ??.

Algorithm 1 Geometry Optimisation Loop

Require: Ordered nodes $\{\mathbf{p}_i\}$, edges, thickness t , flat crush stress σ_{flat}

Require: Learning rate η , number of steps n_{steps} , penalty weights λ_{smooth} , λ_{force} , λ_{radial}

- 1: Compute centroid $\bar{\mathbf{p}}$ and transform to spherical: (r_i, θ_i, ϕ_i)
 - 2: Initialise r_i as learnable parameters; fix θ_i, ϕ_i
 - 3: Compute initial crush force F_{crush}^0 (forward pass with no gradients)
 - 4: Initialise Adam optimiser with learning rate η
 - 5: **for** $t = 1$ to n_{steps} **do**
 - 6: Clear gradients: `optimizer.zero_grad()`
 - 7: **Forward pass:**
 - 8: Reconstruct Cartesian coordinates from (r_i, θ_i, ϕ_i)
 - 9: Extend nodes with overlap for closed-loop spline fitting
 - 10: Compute chord-length parametrisation $\hat{s}$
 - 11: Fit natural cubic spline $\mathbf{C}(\hat{s})$
 - 12: Evaluate $\mathbf{C}'$ and $\mathbf{C}''$ at node locations
 - 13: Compute RoC: $R_i = \|\mathbf{C}'\|^3 / \|\mathbf{C}' \times \mathbf{C}''\|$
 - 14: Compute crush stresses: $\sigma_i = \sigma_{\text{flat}} \cdot f_{\text{RoC}}(R_i)$
 - 15: Compute perimeter P and crush force $F_{\text{crush}} = t \cdot P \cdot \bar{\sigma}$
 - 16: Compute penalties $\mathcal{P}_{\text{smooth}}$, $\mathcal{P}_{\text{force}}$, $\mathcal{P}_{\text{radial}}$
 - 17: Assemble loss $\mathcal{L}$ (Eq. ??, ??, or ??)
 - 18: **Backward pass:** `loss.backward()` $\rightarrow$ compute $\partial \mathcal{L} / \partial r_i$ for all i
 - 19: **Parameter update:** `optimizer.step()` $\rightarrow$ Adam update of r_i
 - 20: **end for**
 - 21: **return** Optimised node positions
-

10 Script Architecture

The script is structured as a PyTorch model class with helper functions. Table ?? maps each function to its mathematical role.

Table 1: Mapping of script functions to mathematical operations

Function	Mathematical Operation	Reference
<code>Cartesian2Spherical()</code>	$(x, y, z) \rightarrow (r, \theta, \phi)$	Eqs. ??–??
<code>Spherical2Cartesian()</code>	$(r, \theta, \phi) \rightarrow (x, y, z)$	Inverse of above
<code>nodesCoords()</code>	Overlap extension for closed loop	Section ??
<code>chordLength()</code>	Chord-length parametrisation $\hat{s}$	Section ??
<code>splines()</code>	Spline fit + derivatives $\mathbf{C}', \mathbf{C}''$	Eq. ??
<code>RoCCalc()</code>	$R_i = 1/\kappa_i$ from 3D curvature formula	Eqs. ??–??
<code>CurvatureEq()</code>	$f_{\text{RoC}}(R) = 7.95 R^{-0.94} + 0.99$	Eq. ??
<code>perimeterCalc()</code>	$P = \sum \ \mathbf{p}_{i+1} - \mathbf{p}_i\ $	Eq. ??
<code>smoothnessTerm()</code>	$\sum (\Delta^2 r)^2$	Eq. ??
<code>crushOptimizerModel.forward()</code>	Full forward pass: $r \rightarrow F_{\text{crush}}, P$	Eq. ??
<code>loadData()</code>	Load MATLAB <code>.mat</code> file	—
<code>orderNodes()</code>	Sort nodes into continuous path	—

10.1 Data Flow Summary

The complete data flow through a single forward pass can be represented as:

$$r_i \xrightarrow{\text{Sph} \rightarrow \text{Cart}} \mathbf{p}_i \xrightarrow{\text{overlap}} \mathbf{p}_i^{\text{ext}} \xrightarrow{\text{chord}} \hat{s} \xrightarrow{\text{spline}} \mathbf{C}', \mathbf{C}'' \xrightarrow{\text{Eq. ??}} R_i \xrightarrow{\text{Eq. ??}} \sigma_i \xrightarrow{\text{Eq. ??}} F_{\text{crush}}$$

Every operation in this chain is differentiable within PyTorch, enabling end-to-end gradient computation from $\mathcal{L}$ back to r_i .

11 Hyperparameters and Configuration

Table ?? lists the default hyperparameters used in the script.

Table 2: Default hyperparameters

Parameter	Symbol	Default Value
Number of iterations	n_{steps}	15 000
Base learning rate	η	0.1
Overlap count	n_{ov}	7
Smoothness weight	λ_{smooth}	10^{10}
Crush force weight	λ_{force}	10^{10}
Radial distance weight	λ_{radial}	10^{10}
Shell thickness	t	2 mm
Flat crush stress	σ_{flat}	90 MPa
RoC lower clamp	—	5 mm